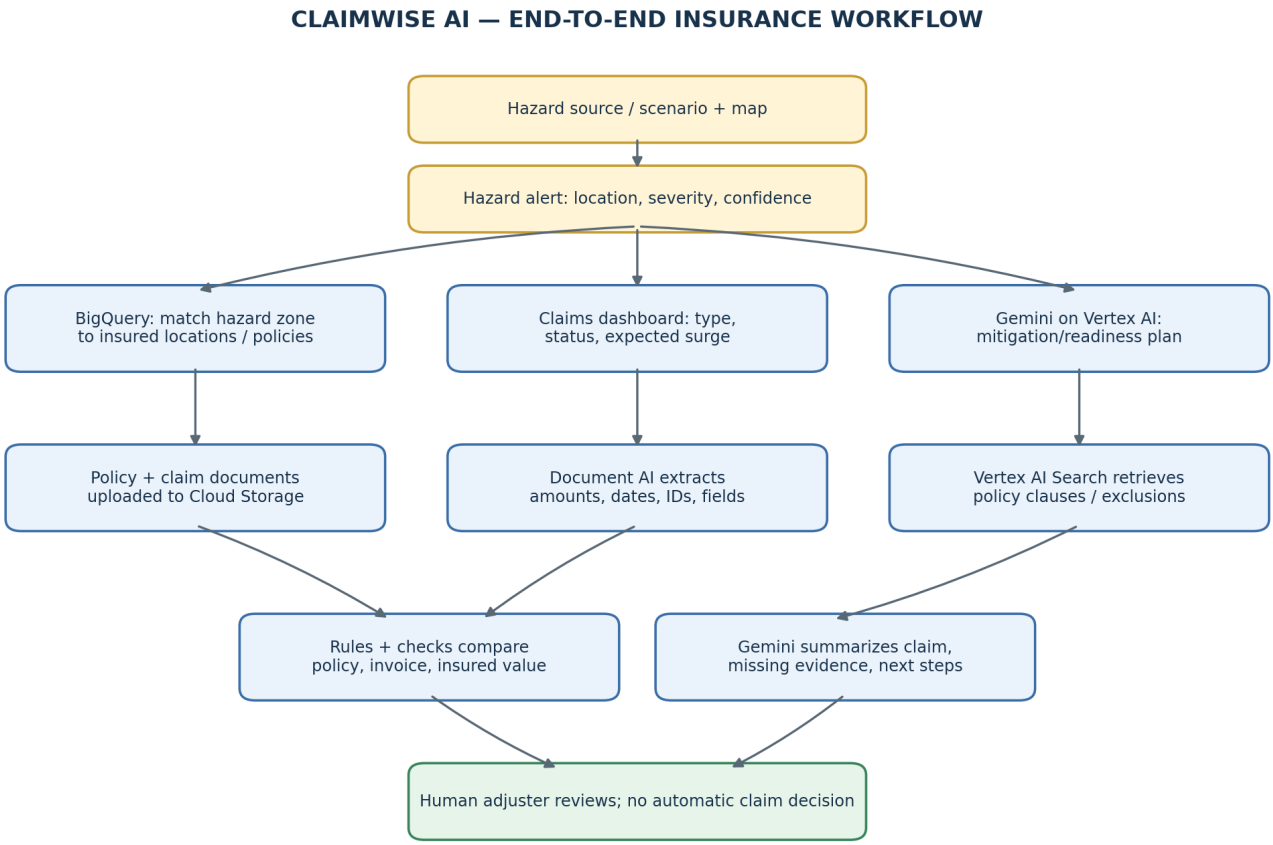
CLAIMWISE AI

Full Project Guide — Google Cloud Hackathon

Insurance catastrophe alerts • Claims operations • Document and policy review



1. Project summary

ClaimWise AI is an insurance operations assistant for catastrophe events such as floods, cyclones and major fires. It connects hazard alerts to an insurer's portfolio, shows which insured properties may be exposed, helps teams prepare for incoming claims, and supports claim-by-claim document and policy review.

The system is decision support, not an automatic claim decision-maker. A hazard alert does not prove damage, and a document mismatch does not by itself prove fraud.

2. The problem we are solving

During a catastrophe, an insurer may receive many claims at once. Teams need to know where the event may affect policyholders, how severe it may be, which claim types are arriving, how to prepare staff, whether documents are complete, whether amounts match, and what the policy wording says is covered or excluded. ClaimWise AI puts those tasks into one workflow.

3. Main website sections

Section A — Catastrophe alerts and severity

• Show a map with an alert area, event type, expected time window, severity level (for example Low, Moderate, High, Severe), and confidence/source timestamp.

• Show the source of the alert. For the hackathon, use a clearly labelled simulated alert unless a verified live data source is actually integrated.

• Overlay synthetic insured properties and policies inside or near the impact area.

• Calculate exposed policy count and insured value using deterministic geospatial rules and BigQuery where feasible.

• Severity must be based on disclosed rules/inputs; do not invent official forecasts or claim certainty.

Section B — Claims command center

• List claims arriving in the affected area and group/filter by claim type: property damage, motor/automobile, fire, theft, casualty/liability, business interruption, or other.

• Show claim ID, policy ID, reported loss amount, claim status, location label, document status, and priority indicator.

• Forecast workload using visible assumptions such as estimated claim rate, average claim cost, available adjusters, and claims handled per adjuster per day.

• Create a mitigation/readiness plan: policyholder outreach, emergency contact prioritisation, adjuster allocation, document checklist and escalation of urgent cases.

Section C — Claim detail and document review

• Allow the user to open a claim and upload documents such as claim form, repair estimate, invoice, receipt, photos, police/fire report, vehicle registration, proof of ownership, or proof of payment.

• Use Document AI to extract available fields: names/IDs, dates, invoice numbers, line items, totals, tax, and other fields supported by the selected processor.

• Compare extracted values with claim data, policy limits, deductibles, and other submitted documents using deterministic code.

• Show a simple money check: reported amount, document total, policy limit, deductible, and any differences. Label this as a preliminary check, not a final settlement amount.

• Check document completeness against a claim-type checklist. State exactly what is missing and why it may be needed.

• Generate a concise claim summary: reported event, claimed amount, documents received, checks completed, issues found, missing items, and next suggested action.

Section D — Coverage and exclusions

• Show policy details: policy type, coverage period, insured item, limit, deductible, covered perils, exclusions and relevant conditions.

• Use Vertex AI Search to retrieve relevant policy clauses and wording. Show the clause text or source reference beside the explanation.

• Organise the result into: 'Potentially covered based on supplied wording', 'Potentially excluded or restricted', and 'Needs human review'.

• Check that the policy was active on the reported loss date and that the event type matches the policy wording, where the necessary data exists.

• Do not state a definitive coverage decision if wording, endorsements, facts, or required evidence are missing. Do not auto-approve or auto-reject a claim.

4. End-to-end flow

• 1. A hazard alert or demo scenario is selected.

2. The alert shows type, area, severity, source and timestamp.

3. BigQuery matches the affected zone with insured locations and policies.

4. The claims dashboard shows incoming claims by type and estimates workload.

5. Gemini on Vertex AI creates a readiness and mitigation plan using verified numbers and stated assumptions.

6. A user opens a claim and uploads documents; Cloud Storage stores the files.

7. Document AI extracts document fields.

8. Deterministic checks compare amounts, dates, policy limits and document completeness.

9. Vertex AI Search retrieves relevant policy clauses and exclusions.

10. Gemini writes a grounded claim summary and explains issues/missing evidence, with source references where available.

11. A human adjuster reviews the findings and decides the next action.

5. Recommended Google Cloud tech stack

6. Application tech stack

• Frontend: Next.js, React, TypeScript, Tailwind CSS, Leaflet (or MapLibre) and Recharts.

• Backend: Python FastAPI on Cloud Run; use REST APIs with JSON request/response bodies.

• Data: BigQuery for analytics; JSON/CSV seed data for the fast MVP; Cloud Storage for documents.

• AI: Gemini on Vertex AI, Document AI, Vertex AI Search.

• Version control: GitHub with feature branches and pull requests.

• Security: server-side secrets, least-privilege service accounts, upload size/type checks, and synthetic demo data.

7. Data we need

For the hackathon, use fictional policyholders and locations. Start with 20–100 synthetic policies, 8–15 claims, 3–5 sample policy documents, and 3–6 sample claim document files. You do not need a huge dataset to demonstrate the workflow.

8. Dummy data starter

A separate JSON file is provided with a simulated cyclone scenario, three fictional policies, three fictional claims, sample document metadata and editable workload assumptions. Expand it with more records if needed. All values are fictional and are for testing only.

Example demo records:

• POL-A101: Commercial Property, illustrative limit ■50,00,000, covered perils include cyclone/windstorm/fire subject to wording.

• CLM-2001: Property damage after simulated cyclone; reported loss ■4,80,000; final invoice missing.

• CLM-2002: Motor claim after simulated flood; reported loss ■1,75,000; vehicle registration copy and damage photos missing.

• CLM-2003: Fire damage; reported loss ■3,20,000; proof of ownership/insurable interest missing.

These examples do not establish real policy coverage. The wording and claim facts are deliberately simplified; the application must show uncertainty and request human review.

9. Three-person work split

Person 1 — You + Codex: Frontend and end-to-end integration

Build the map and catastrophe alert panel; severity badge; claims dashboard and claim-type filters; claim detail/upload UI; coverage/exclusions panel; connect APIs; polish and rehearse the demo.

Person 2 — Teammate + Claude: Backend, BigQuery and calculations

Create the synthetic datasets and BigQuery tables/queries; implement map-zone-to-policy exposure matching; claim list and claim-type grouping; money/limit/deductible checks; workload estimates; API endpoints and Cloud Run deployment.

Person 3 — Teammate + Claude: AI and documents

Set up Gemini on Vertex AI; Document AI extraction for one or two sample document types; Vertex AI Search for policy clauses; prompts for claim summaries, missing-document explanations and mitigation plan; source-grounding and fallback sample results.

Shared contract on Day 1: agree on JSON fields and endpoint names. Frontend can use mock responses until the API is ready. One person owns deployment. Commit small changes often.

10. API endpoints to agree on

11. Overnight implementation plan

12. MVP priorities

• P0: Map alert with simulated/live label; policy exposure count; claims list by type; claim detail; BigQuery; Gemini readiness plan; Cloud Run deployment.

• P1: One claim document upload, Document AI extraction, missing-document and money checks, Vertex AI Search policy clauses, grounded summary.

• P2: Pub/Sub alert automation, many document types, complex policy endorsement reasoning, real-time external hazard data, advanced forecasting.

13. Important rules and limitations

• Never label a simulated event as an official warning. For real alerts, show the actual source and timestamp.

• Map exposure means a policy may be in the affected area; it does not prove damage or coverage.

• Use deterministic code for arithmetic, policy dates, totals, deductibles and document completeness. Use AI to explain, summarise and suggest next steps.

• Coverage wording can depend on endorsements, exclusions, deductibles, conditions and facts not yet provided. Show 'potentially covered', 'potentially excluded/restricted', or 'needs human review'.

• Do not auto-approve or auto-reject claims and do not label discrepancies as fraud without evidence.

• Never put credentials in frontend code. Use server-side environment variables and least-privilege service accounts.

14. Final demo script

1) Open a simulated catastrophe alert and explain its severity/source. 2) Show exposed insured properties and estimated insured value. 3) Filter incoming claims by property, motor, fire or theft. 4) Open one claim and upload a sample repair estimate. 5) Show extracted amounts and any mismatch or missing documents. 6) Show policy clauses grouped into potential coverage, potential exclusions/restrictions, and needs review. 7) Show the claim summary and mitigation/next-step plan. 8) Explain that an adjuster makes the final decision.

15. Short project pitch

ClaimWise AI is a Google Cloud-powered insurance catastrophe and claims assistant. It connects hazard alerts to an insurer's policy portfolio, helps teams prepare for claim surges, and reviews claim documents against policy wording to surface missing evidence, amount mismatches, potential coverage and exclusions for human review.

Technology	Purpose
Cloud Run	Host the API and, if practical, the web application.
BigQuery	Analyse portfolio, policies, locations, exposure and claim workload.
Gemini on Vertex AI	Create mitigation plans and claim summaries from provided facts; explain findings without inventing values.
Document AI	Extract fields from uploaded claim documents.
Vertex AI Search	Search policy wording, endorsements, exclusions and claim procedures.
Cloud Storage	Store uploaded documents and demo data files.
Pub/Sub (optional)	Publish a new catastrophe alert event and trigger follow-up processing.
Cloud SQL / AlloyDB (optional)	Store operational claim state if needed; skip if it adds unnecessary setup.
Maps: Leaflet + OpenStreetMap or permitted mapping service	Render the map and markers; mapping is separate from hazard truth.

Data group	Fields to include
Hazard alert/scenario	event_id, hazard_type, region, geometry or radius, severity inputs, expected time, source, timestamp, confidence, simulated/live flag.
Insured portfolio/policy	policy_id, customer_id, insured_item_id, insured type, location coordinates, policy type, effective dates, coverage limit, deductible, covered perils, exclusions, endorsements.
Claims	claim_id, policy_id, reported date, loss date, loss cause, claim type, reported amount, status, location, assigned adjuster.
Documents	document_id, claim_id, file path, document type, upload time, extracted fields, extraction confidence, validation status.

Data group	Fields to include
Policy wording	policy document ID, clause heading, clause text, coverage condition, exclusion, endorsement, source page/section.
Operational assumptions	estimated claim rate, average claim cost, adjuster capacity, claims handled per adjuster per day, escalation thresholds.

Endpoint	Purpose
GET /api/alerts	Return hazard alerts with severity, source, timestamp and simulated/live flag.
GET /api/alerts/{id}/exposure	Return policies and insured value in the affected zone.
GET /api/claims?type=property_damage	List claims, filter by type/status/region.
GET /api/claims/{id}	Return claim details, policy details, documents and status.
POST /api/claims/{id}/documents	Upload or register a claim document.
POST /api/claims/{id}/analyze	Run extraction, amount checks, completeness checks and policy retrieval.
GET /api/claims/{id}/summary	Return the summary, potential coverage/exclusion findings and missing evidence.
POST /api/readiness-plan	Create a mitigation/readiness plan from alert and exposure results.

When	Target
First 30 minutes	Freeze scope, check GCP permissions/APIs/billing, create repo and agree on data/API contracts.
Next 2 hours	Build alert/map, claims list, claim detail screens; seed JSON data; backend exposure query and claim APIs.
Next 1–2 hours	Connect BigQuery and Gemini readiness plan; verify the numbers and assumptions.
Next 1–2 hours	Add one document upload and Document AI extraction; add Vertex AI Search policy retrieval if working.
Next 1 hour	Add amount/document completeness checks, summary and coverage/exclusion layout.
Before sleep / feature freeze	Deploy Cloud Run, test one full workflow, record backup screenshots/video, stop adding features.
Tomorrow morning	Fix high-impact bugs, verify permissions, rehearse, prepare a 2-minute explanation and submit before 1 PM.